# Windows run guide

## Repo3 R3-Budget and Repo4 SeqMap

Prepared 13 September 2026. Use Python 3.12 (64-bit), Git, and VS Code on Windows. CPU execution is supported; neither project requires a GPU. BDF reading requires the optional MNE dependency.

Your dataset stays at:

```
D:\EEG_Dataset\HBN_100Hz
```

Expected folders: R4_L100_bdf, R5_L100_bdf, R7_L100_bdf, R9_L100_bdf.

Repo3 studies reliability-guided GAN augmentation and selective clean-data budgets. Repo4 studies sequential feature combinations and heatmaps. You can prepare the EEG and extract features once, then use the same feature CSV in both projects.

The supplied files contain a protocol and source code, not EEG recordings or a clinical table. Folder names alone cannot determine diagnoses, EO/EC events, electrode mapping or previous preprocessing. The preparation steps below make those requirements explicit. Do not populate them with guessed values.

All shown synthetic results are software demonstrations. The manuscripts contain no fabricated HBN classification results. A target of 90% accepted-case performance is a tuning goal for Repo3's selective policy, not a promised result or publication acceptance rate.

Use a separate virtual environment for each repository. Run commands from the repository folder specified on each page. Keep local data and results in ignored local/ and outputs/ folders. Do not add EEG or clinical records to GitHub.

# 1. Clone the repositories and install

Create a work folder, for example D:\EEG_Projects. Open PowerShell there. Sign in to GitHub if Git prompts, because the repositories are private.

```
New-Item -ItemType Directory -Force D:\EEG_Projects
Set-Location D:\EEG_Projects
git clone https://github.com/Mj-heidari/Repo3.git
git clone https://github.com/Mj-heidari/Repo4.git
Set-Location Repo3
py -3.12 -m venv .venv
$py = ".\.venv\Scripts\python.exe"
& $py -m pip install --upgrade pip
& $py -m pip install -e ".[bdf]"
& $py -m hbn_r3.cli --help
```

If py -3.12 fails, install Python 3.12 (64-bit) from python.org and reopen the terminal. Git is available from git-scm.com. This guide uses the environment's Python executable directly, so activating a PowerShell script is unnecessary.

Open D:\EEG_Projects\Repo3 in VS Code. Select Python: Select Interpreter and choose Repo3\.venv\Scripts\python.exe. The included VS Code settings point to this environment.

Install Repo4 separately:

```
Set-Location D:\EEG_Projects\Repo4
py -3.12 -m venv .venv
$py = ".\.venv\Scripts\python.exe"
& $py -m pip install -e ".[bdf]"
```

MNE is only needed for BDF I/O. If you already have audited NPZ epochs, pip install -e . is sufficient for classification, GAN training and visualization. Use each repository's own environment because they share a helper-package name.

# 2. Verify Repo3 before touching real data

From Repo3, run the tests and the short synthetic classifier experiment:

```
Set-Location D:\EEG_Projects\Repo3
$py = ".\.venv\Scripts\python.exe"
$env:OPENBLAS_NUM_THREADS = "1"
& $py -m unittest discover -s tests -v
& $py -m hbn_r3.cli demo --output demo_data/features.csv
& $py -m hbn_r3.cli run `
  --features demo_data/features.csv `
  --config configs/smoke.json --output outputs/smoke_01
& $py -m hbn_r3.cli plot --results outputs/smoke_01
```

Expected: 19 tests pass; the experiment prints each held-out release and model. It writes predictions, metrics, split records, model files and augmentation diagnostics. The duration plot is marked SYNTHETIC SOFTWARE CHECK. Do not copy synthetic metrics into either paper.

Check the selective policy separately:

```
& $py -m hbn_r3.cli run `
  --features demo_data/features.csv `
  --config configs/budget_smoke.json `
  --output outputs/budget_smoke_01
```

Expected files include budget_predictions.csv, budget_metrics.json and policy_*.json. An infeasible policy may accept nobody; that is an explicit outcome, not a crash. Ordinary fixed-budget predictions are still reported.

Choose a fresh output directory for every run. The code refuses to overwrite a nonempty result directory. Changing _01 to _02 is enough. The short GAN smoke uses four generator steps only; it checks execution, not GAN convergence.

# 3. Inspect your actual BDF collection

Inventory the dataset without modifying it:

```
& $py -m hbn_r3.cli inspect-data `
  --root "D:\EEG_Dataset\HBN_100Hz" `
  --output local/inventory.csv
```

Open local/inventory.csv. Find a representative BDF path. Replace REAL_FILE.bdf below with its actual filename:

```
& $py -m hbn_r3.cli inspect-bdf `
  --input "D:\EEG_Dataset\HBN_100Hz\R4_L100_bdf\REAL_FILE.bdf" `
  --output local/inspect_one
```

Read local/inspect_one/bdf_info.json for channel names, types, sampling rate and length. Read annotations.csv for embedded descriptions. Repeat for representative files from all four releases and any distinct montage/file type.

If the recording uses a trigger channel, identify its real name from metadata and inspect it explicitly. Status below is only an example, not an assertion about your data:

```
& $py -m hbn_r3.cli inspect-bdf `
  --input "D:\EEG_Dataset\HBN_100Hz\R4_L100_bdf\REAL_FILE.bdf" `
  --output local/inspect_one --stim-channel Status
```

This writes trigger_events.csv. The default event bit mask is 65535; verify the acquisition encoding before using that mask or overriding --event-mask. Never assume that trigger 1 means eyes open or trigger 2 means eyes closed. Match codes or annotations to the actual paradigm documentation and recording.

If you cannot establish EO/EC intervals, pause real-data extraction. Preserve the inventory and de-identified event/header information so the adapter can be matched to the actual schema. Do not label an entire rest file EO or EC just from its folder name.

# 4. Build the channel, event and clinical inputs

Create local/channels.json as a JSON list of verified EEG channel names. Use the labels exactly as returned by inspection; exclude trigger and non-EEG channels. The empty example template is deliberately invalid until filled.

Create local/regions.json with exactly six nonempty, disjoint lists for left frontal, right frontal, central, parietal, left temporal and right temporal regions. Use the actual electrode montage. The supplied 10-20 example demonstrates syntax only and must not be treated as an EGI mapping.

For each BDF, create an interval CSV with this header:

```
onset_seconds,duration_seconds,condition
```

Fill it from verified event timing. Conditions are eo or ec. Times are seconds from the beginning of that recording. Intervals must be positive, nonoverlapping and inside the file. The code trims two seconds from both ends by default; accepted epochs never cross a condition boundary.

Create local/raw_manifest.csv with the header in examples/raw_manifest_SCHEMA_ONLY.csv. One row represents one participant:

- participant_id: globally unique clinical ID, not a newly invented ID per file.
- release: R4_L100_bdf, R5_L100_bdf, R7_L100_bdf or R9_L100_bdf.
- path: absolute BDF path, or a path relative to this manifest.
- intervals_path: that recording's verified interval CSV path.
- label: 1 for confirmed ADHD, 0 for the explicitly defined control group.
- age and sex: metadata from the phenotype record.
- group_id: shared family identifier; use participant ID only if family information is unavailable and disclose that limitation.
- diagnosis_confirmed: true only after confirming the label/control definition.
- cleaning_provenance: known source filtering, resampling and cleaning history.

A missing diagnosis is never label 0. Resolve duplicated participants before analysis. Do not treat the release folder as an independent clinical site.

# 5. Prepare and audit the EEG epochs

From Repo3, after reviewing the preprocessing choices:

```
& $py -m hbn_r3.cli prepare-cohort `
  --manifest local/raw_manifest.csv `
  --channels local/channels.json `
  --output-dir local/prepared `
  --reference average --filter-mode bandpass `
  --confirm-preprocessing
```

This example requests an average reference and a 1-40 Hz bandpass. Use --reference keep and/or --filter-mode keep only when the existing derivative reference/filtering is documented and appropriate. Do not refilter or rereference blindly.

Default behavior: fourth-order Butterworth filtering when requested; resampling to 100 Hz if the source is higher; four-second epochs; two-second interval trimming; rejection for BAD annotations, peak-to-peak amplitude above 150 microvolts or flatness below 0.1 microvolts in any selected channel. Thresholds must be fixed after a diagnosis-blinded pilot, not altered to improve classification.

The command creates record_NNNNN.npz files, per-record epoch logs and provenance JSON, preparation_exclusions.csv, and a prepared manifest. The output manifest updates after every successful participant. Use a fresh output directory for a new preparation run.

Inspect representative EEG traces and rejection patterns before analysis. This helper does not perform ICA or channel interpolation and cannot repair previously aliased data. The confirmation flag records that you reviewed the preprocessing policy; it does not certify the EEG automatically.

Validate the prepared cohort and actual regions:

```
& $py -m hbn_r3.cli validate-data `
  --manifest local/prepared/manifest.csv `
  --regions local/regions.json `
  --output local/validation.csv
```

Review exclusions by release and diagnosis. The primary cohort requires age 6-17 and at least 60 accepted seconds in each condition. Do not quietly weaken the rules for the harder class.

# 6. Extract the common feature table

Extract once and reuse the same table in both projects:

```
& $py -m hbn_r3.cli extract `
  --manifest local/prepared/manifest.csv `
  --regions local/regions.json `
  --output local/features.csv
```

Expected outputs are local/features.csv, features.excluded.csv and features.audit.json. A fully included participant has three duration rows: 20, 40 and 60 clean seconds per condition. Each row contains full features (f__), first-half features (a__), second-half features (b__), and metadata.

There are 315 full candidates: regional spectral features, interregional debiased squared wPLI, regional strengths, and EO/EC contrasts. The two half estimates support reliability analysis and the agreement policy. Shorter duration subsets are nested inside a fixed 60-second anchor. They are not chronological acquisition prefixes.

If no participant is eligible, read the exclusion log. Common causes are insufficient clean EO duration, an incorrect regional mapping, missing age or wrong array dimensions. The extractor intentionally refuses to guess units or condition axes.

Alternative input: already audited MAT epochs can be converted with convert-mat after inspecting the actual variable names and axes. Use inspect-mat --input FILE first. The converter needs EO/EC variable names, sampling metadata key, explicit e/c/t axis order, channel names and scale-to-volts. MAT v7.3 needs a dedicated HDF5 adapter or an upstream export to the documented NPZ/MAT format; arbitrary continuous matrices are not silently reshaped into epochs.

The NPZ contract is eo/ec arrays shaped epochs x channels x 400, sfreq=100, ch_names, units="V" and a scalar boolean cleaned=true. The clinical manifest remains separate from the signal arrays.

# 7. Run Repo3 research experiments

From Repo3, use the real extracted feature table:

```
& $py -m hbn_r3.cli run --features local/features.csv `
  --config configs/research.json --output outputs/research_01
& $py -m hbn_r3.cli run --features local/features.csv `
  --config configs/gan_research.json --output outputs/gan_01
& $py -m hbn_r3.cli run --features local/features.csv `
  --config configs/budget_research.json --output outputs/budget_01
```

The first run compares feature-selection variants plus RF, RBF-SVM and logistic regression. The second compares no augmentation, duplication, jitter, unguided GAN, support-filtered GAN and reliability-guided GAN. A zero augmentation ratio is allowed. The third compares margin-only and split-view agreement policies for unaugmented/guided RF.

For demographic adjustment, run the same feature table using configs/research_adjusted.json and a new result directory. Prespecify repeat seeds, for example 2026, 2027 and 2028, by copying the configuration and changing seed before inspecting results. Keep every seed result.

Read release_metrics.csv before pooled_metrics.json. Read paired_comparisons.json for paired changes and conditional family-bootstrap intervals. The SVM score is a margin, not a probability. Inspect augmentation_*.json for rejected candidates and unstable training. Zero accepted synthetic rows is a reportable outcome.

For the evidence policy, read budget_metrics.json together with budget_predictions.csv. Report coverage, class-specific coverage, accepted-case accuracy/balanced accuracy and mean clean seconds per condition. Report ordinary all-participant performance alongside selective performance. Never call 95% accuracy at low coverage "95% accuracy on all participants."

The GAN operates on selected tabular features, not raw EEG. Its support check does not prove physiological realism or privacy. The 90% inner policy target does not guarantee 90% on a new release. Research runs can be substantially slower than smoke runs; no fixed runtime is assumed.

# 8. Run Repo4 sequential combinations

Switch to Repo4 and its environment. Test the complete synthetic path first:

```
Set-Location D:\EEG_Projects\Repo4
$py = ".\.venv\Scripts\python.exe"
& $py -m unittest discover -s tests -v
& $py -m hbn_r4.cli demo --output demo_data/features.csv
& $py -m hbn_r4.cli search `
  --features demo_data/features.csv `
  --config configs/smoke.json --output outputs/smoke_01
& $py -m hbn_r4.cli plot --results outputs/smoke_01
```

Now run the primary real-data block search using Repo3's feature table:

```
& $py -m hbn_r4.cli search `
  --features ../Repo3/local/features.csv `
  --config configs/research_blocks.json `
  --output outputs/research_blocks_01
& $py -m hbn_r4.cli plot --results outputs/research_blocks_01
```

The 12 candidates are six regional spectral blocks, three connectivity-band blocks and three strength-band blocks. Each forward step evaluates all remaining additions inside the outer training data. A one-SE heuristic chooses a compact combination from the trajectory.

Optional: research_bands.json searches 36 individual static band-power features. research_features_expensive.json searches all 315 individual features. The latter requires 11,142 candidate forest fits per outer release before final comparator fits; it can be expensive on a laptop. Start with blocks and prespecify alternative searches rather than selecting a favorable outer result afterward.

Read selected_panels.json for the exact selected group and feature lists for each held-out release. There is no claimed globally optimal or universal best combination. The selected, first-step and all-candidate models are compared in metrics.csv and paired_comparisons.json.

# 9. Read and export the heatmaps

Repo4 writes six figure families in the run's figures folder, each as PDF and PNG:

- panel_selection_heatmap: binary inclusion of each block/candidate in each outer panel.
- co_selection_heatmap: how often candidate pairs coexist; not a biological interaction matrix.
- conditional_gain_heatmap: improvement in mean inner score for each next addition, with the chosen addition outlined. Blank cells are not evaluated at that step.
- selection_paths: panel size versus inner score; the chosen compact panel is circled.
- held_out_performance: outer scores for the selected, first-step and full candidate models.
- individual_feature_heatmap: up to 30 most frequently selected coordinates.

Large feature searches show a readable candidate subset; complete evaluations remain in candidate_evaluations.json. The conditional-gain color scale is fixed to +/-0.15; values outside it saturate, while exact values remain available in the JSON.

feature_frequency.csv and panel_jaccard.csv provide machine-readable selection summaries. Four outer panels give only coarse stability estimates. Correlated alternatives can substitute for one another without changing accuracy, and co-selection frequency is not causal synergy.

For the Repo4 paper, use the PDF versions of actual real-data plots. Synthetic figures are included separately as preview examples and have prominent watermarks. Do not remove the watermark and present them as empirical HBN results.

For the Repo3 paper, use actual classification, augmentation and evidence-policy outputs. The current draft includes a methods diagram and an honest results-status section. It will need a real cohort/results section after the experiments finish.

# 10. Rebuild papers and troubleshoot

Each repository contains paper/main.tex, references.bib, bibliography.tex, IEEEtran.cls and main.pdf. The separate LaTeX archive contains both projects' sources. Open main.tex in Overleaf or use a local TeX installation with the required common packages.

```
# Run inside the desired repository:
& $py scripts/render_bibliography.py
Set-Location paper
pdflatex -interaction=nonstopmode -halt-on-error main.tex
pdflatex -interaction=nonstopmode -halt-on-error main.tex
Set-Location ..
```

The controlled bibliography renderer writes bibliography.tex from cited BibTeX entries, so this build does not require IEEEtran.bst. Author details, affiliations, empirical results and the target conference's exact submission requirements remain to be completed.

Troubleshooting:

- Repository not found: sign in to the GitHub account that has access to these private repositories.
- No module named hbn_r3/hbn_r4: use the correct repository's .venv Python and rerun pip install -e . there.
- No module named mne: install the [bdf] extra in that environment.
- Missing channel/event labels: inspect the actual file; do not substitute guessed mappings.
- Too few clean seconds: inspect the exclusion log and actual condition durations; do not change clinical labels or inflate epoch counts.
- Missing class or insufficient class/release cells: audit the cohort/family groups. Three inner folds and augmentation need adequate data in every cell.
- Output directory not empty: choose a new named run directory; earlier results are preserved.
- All selective predictions abstain: inspect the inner policy feasibility and report that the target was not achieved.
- Very slow full-feature search: use the prespecified block experiment first; keep the costly individual-feature search as a separate sensitivity run.

Local validation covered Python/Linux scientific tests, complete synthetic experiments, and rendered PDFs/figures. Real BDF disk I/O and Windows PowerShell were not executed in this environment. Validate representative BDF files on your device before the full cohort run.